# ECE 310: Project 2 Report
# Design of Complex Digital Systems

Ezana Enquobahrie

March 19, 2026

## Abstract

This report covers the design of a digital arithmetic circuit that computes the function $(A-B)+(C-D)$. The system is implemented in Verilog and follows a strict Controller/Datapath separation. The hardware is built using structural modeling. At the base level, 1-bit full adders are used to construct 8-bit ripple-carry subtractors and a final 9-bit ripple-carry adder. Structural multiplexers are also used to manage data loading into the D-Flip-Flop registers, ensuring the design stays within hardware constraints. Final simulations were verified against the provided testbenches, achieving 100% functional accuracy and meeting all project requirements.

## 1 Introduction, Background, and Theory

The goal of this project was to build a circuit using different levels of Verilog abstraction: structural, dataflow, and behavioral. The system had to take in four 8-bit inputs ($A$, $B$, $C$, and $D$) from a shared bus, store them, and calculate a final result. A major constraint was that behavioral code (always blocks) could only be used for the basic D-Flip-Flop (DFF) component.

Because the DFFs have a free-running clock, a data-recycling loop was used to keep the registers from changing values at the wrong time. 2-to-1 muxes were placed in front of each register; when an operand isn't being loaded, the mux feeds the register's own output back into the input to hold the data.

All arithmetic units were built structurally instead of using the + or - operators. A 1-bit full adder was created using dataflow modeling, and these were then chained together to form an 8-bit Ripple Carry Subtractor and a 9-bit Ripple Carry Adder. Subtraction was handled using 2's complement logic by inverting the bits with structural NOT gates and setting the initial carry-in to 1.

## 2 Results & Analysis

The design was broken down into smaller, hierarchical blocks to make it easier to manage. Table 1 shows the full structure of the Verilog code and what each specific component is responsible for.

| Module Name | Category | Functional Description |
|---|---|---|
| `project2` | Top-Level | Instantiates the Controller and Datapath, routing data and control signals between them. |
| `controller` | Control Logic | Decodes the 2-bit `op` code to assert the correct `load` signal when `capture` is High. Tracks received operands and asserts `valid`. |
| `datapath` | Datapath | Houses the multiplexers, storage registers, and structural arithmetic units to compute $(A - B) + (C - D)$. |
| `RCA8` & `RCA9` | Arithmetic | Structural 8-bit ripple-carry subtractors and a 9-bit ripple-carry adder. Implements 2's complement logic via NOT and XOR gates. |
| `m8`, `r8`, `r10` | Routing/State | 8-bit multiplexers for data recycling loops, and arrays of 1-bit DFFs for synchronous state storage. |
| `my_dff` & `f_add` | Base Logic | The lowest-level building blocks. `my_dff` is the only module using behavioral Verilog; `f_add` uses dataflow logic. |

Table 1: Summary of the structural Verilog modules and their operational responsibilities within the system.

Figure 1 shows the top-level schematic from Vivado RTL Analysis. It shows the separation between the Controller (handling states and load signals) and the Datapath (handling storage and math).

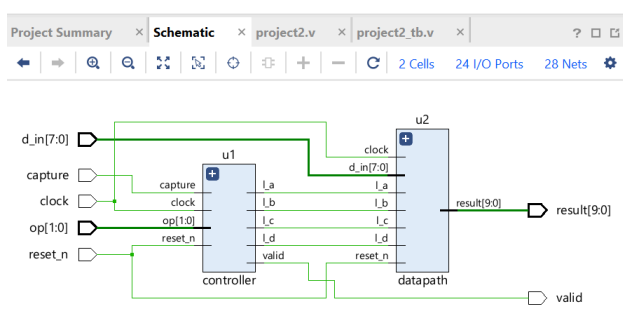


Figure 1: Vivado Elaborated RTL schematic of the Top-Level Project 2 Architecture.

The Controller logic (Figure 2) uses the `capture` and `op` inputs to decide which `load`

signals to send. It uses four DFFs to track which operands have been saved. When all four flags are High, an AND gate triggers the `valid` signal, which is delayed by one cycle so it stays in sync with the final output.

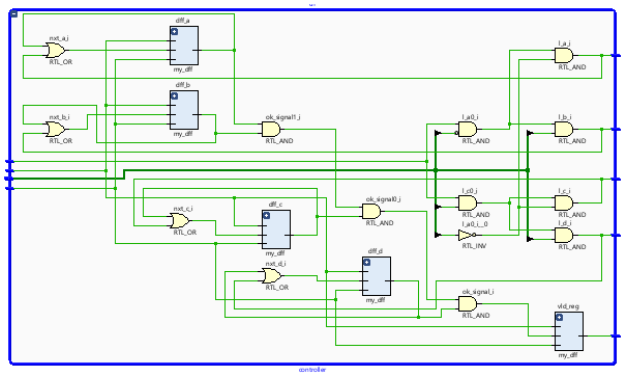

Figure 2: Controller RTL Schematic detailing the load decoding and DFF flags.

The Datapath (Figure 3) routes data through the registers and into the arithmetic blocks. This confirms that the MUX recycling loops are working correctly to hold the data.

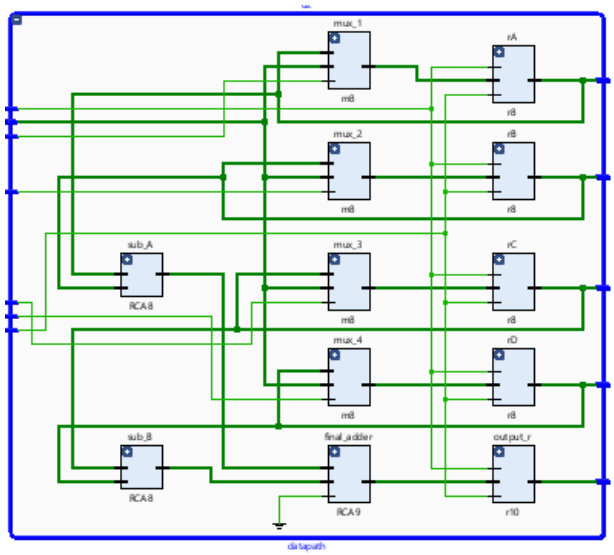


Figure 3: Datapath RTL Schematic showing structural storage and arithmetic routing.

Figures 4 and 5 show the structural logic for the arithmetic blocks. I used XOR gates (`assign d[8] = d[7] ^ c[8] ^ c[7];`) to handle the sign bit extension. This avoids overflow errors during the 2's complement subtraction and ensures negative results are calculated correctly before they enter the final RCA9 adder.

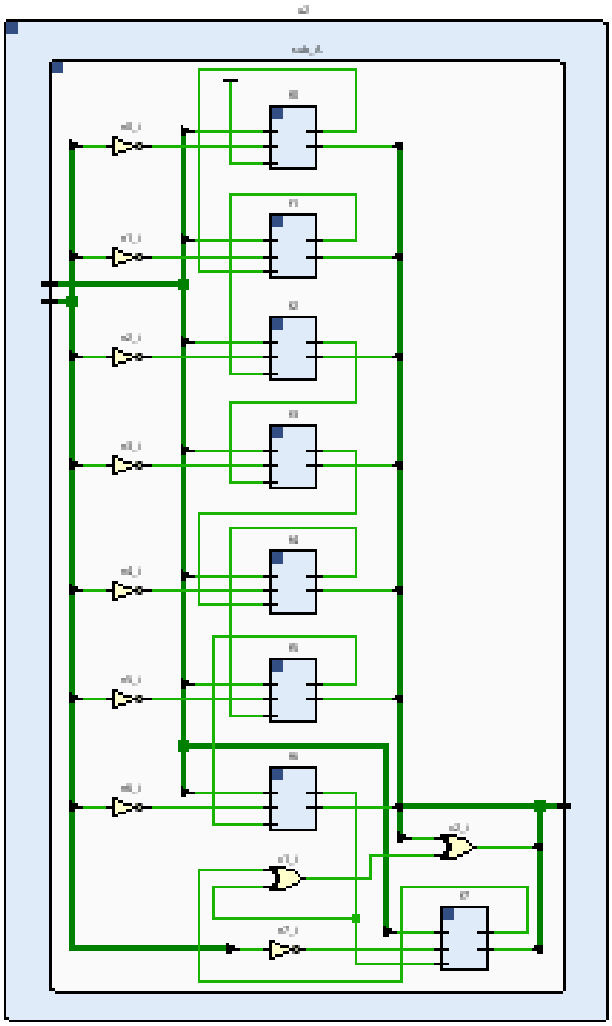

Figure 4: Internal structural logic of the RCA8 Subtractor.

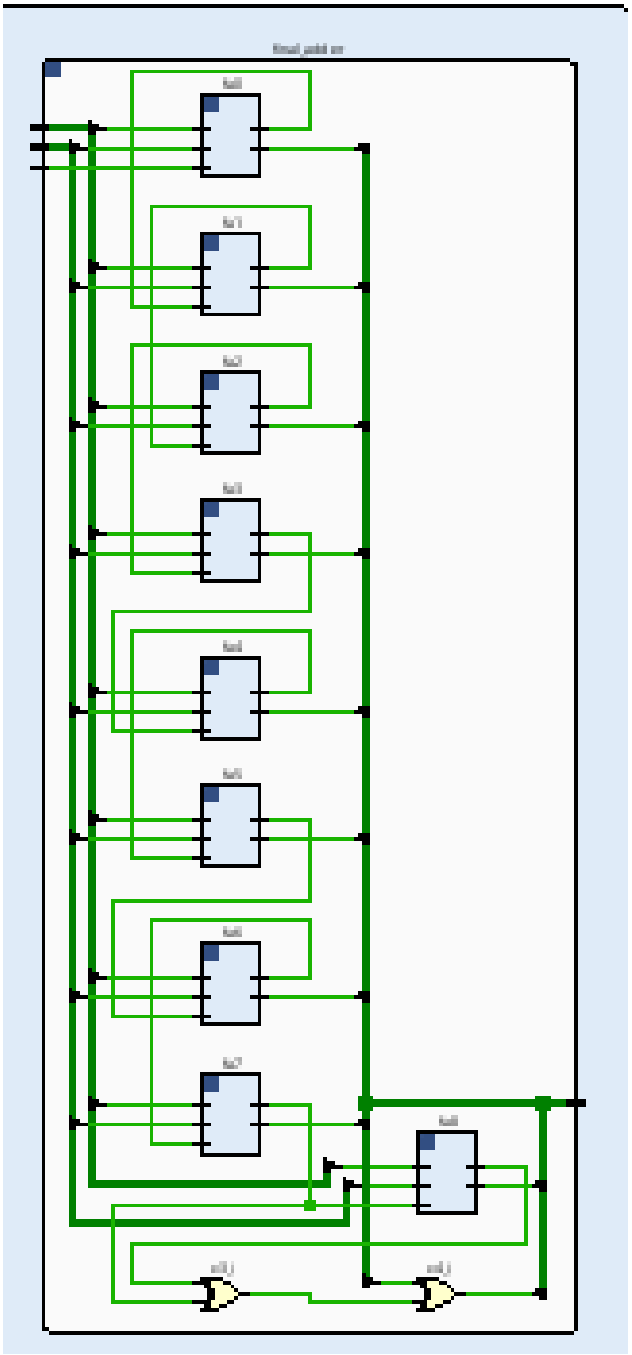

Figure 5: Internal structural logic of the RCA9 Adder.

Simulation in the Verilog testbench confirms that the timing and logic are correct. Figure 6 shows the reset sequence and the four operands being loaded one after another.

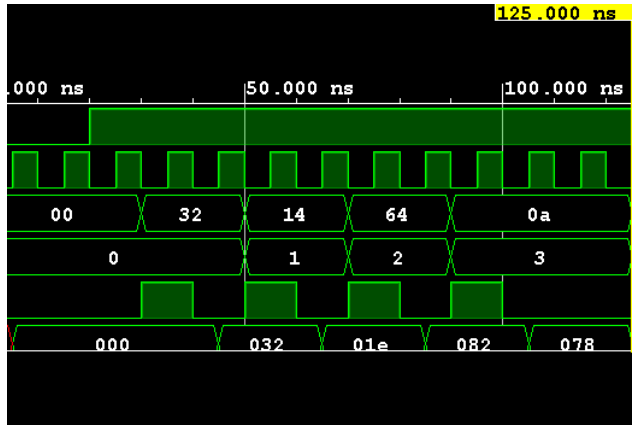


Figure 6: Full simulation waveform overview.

As seen in Figure 7, the mux feedback loops work as expected. When `capture` is High for a specific `op` code, only that register updates, while the others stay the same.

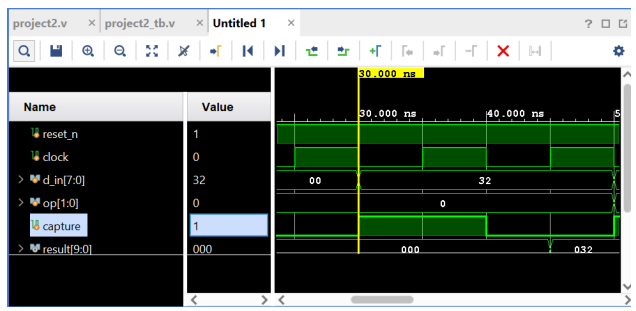


Figure 7: Zoomed waveform displaying the capture sequence for operand A.

Figure 8 shows the timing for the output. The calculation goes through the structural adders combinationally, and the final result is latched into the output register at the same time the `valid` signal goes High.

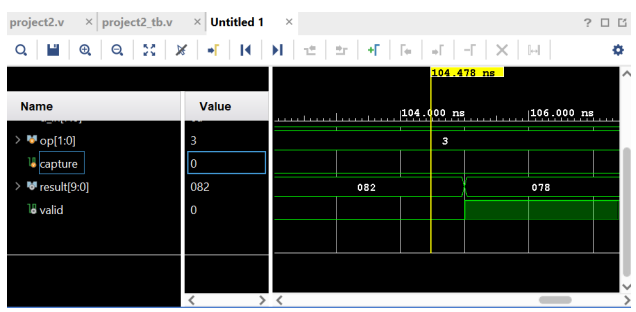


Figure 8: Zoomed waveform showing valid assertion and synchronous result update.

## 2.1 Test Vector Justification

To verify the structural integrity of the arithmetic units, four specific test vectors were chosen: 32 ($A$), 14 ($B$), 64 ($C$), and 0A ($D$). These vectors were selected to ensure the system accurately calculates intermediate results for both $(A - B)$ and $(C - D)$ within the 8-bit subtractors before summing them into the 10-bit output. Since the project requires that the final result remains non-negative, these values provide a clear path for verification: $(32 - 14) + (64 - 10) = 18 + 54 = 72$. This sequence successfully tests the controller's ability to differentiate operands on the shared bus and the datapath's ability to maintain storage across asynchronous load delays.

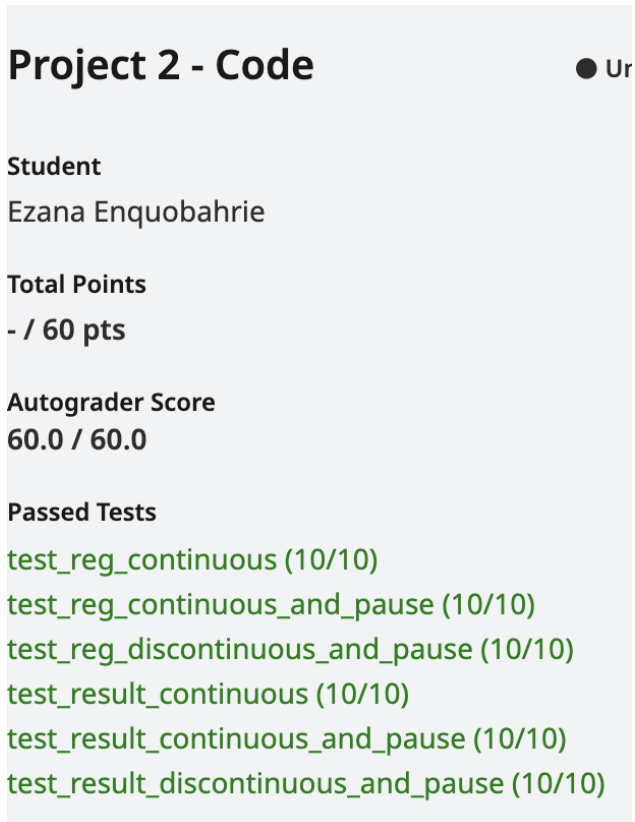


Figure 9: Gradescope autograder output showing the final 60.0/60.0 functional score.

# 3 Conclusions

This project showed how to take a math function and turn it into a purely structural circuit. Translating a high-level equation into fundamental logic gates was challenging but provided a much deeper understanding of digital design. Building the arithmetic units from 1-bit adders proved that signed math works even without using behavioral operators like + or -. By manually handling the 2's complement inversion and sign-bit extension, it became clear how processors actually compute negative values at the hardware level. The implementation of the mux feedback loops also verified that DFFs can be controlled to hold their state

without needing internal always block modifications. This was a crucial workaround that allowed the system to remain synchronous with the free-running clock while strictly adhering to the project's behavioral modeling constraints. If I were to do this again, I would look into using a carry-lookahead adder to make the timing faster compared to the ripple-carry design used here. While the current design is functionally accurate, optimizing the critical path delay will be the next logical step when designing larger and more complex datapaths in the future.

*This report was typeset in LaTeX.*